

T. VIKAS

2303A52281

740. Delete and Earn

class Solution:

```
def deleteAndEarn(self, nums: List[int]) -> int:
```

```
    n = len(nums)
```

```
    if n <= 2:
```

```
        return max(nums)
```

```
    dp = [0] * (n)
```

```
    dp[0] = nums[0]
```

```
    dp[1] = max(nums[0], nums[1])
```

```
    for i in range(2, n):
```

```
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])
```

```
    return dp[n-1]
```

416. Partition Equal Subset Sum

class Solution:

```
def canPartition(self, nums):
```

```
    s = sum(nums)
```

```
    if s%2 == 1:
```

```
        return False
```

```
    target = s//2
```

```
    dp = [False] * (target + 1)
```

```
    dp[0] = True
```

```
    for num in nums:
```

```
        for i in range(target, num - 1, -1):
```

```
            dp[i] = dp[i] or dp[i - num]
```

```
return dp[target]
```

494. Target Sum

```
class Solution:
```

```
def solve(self, i, target, nums, dp):
```

```
    if i == 0:
```

```
        if nums[0] == 0 and target == 0:
```

```
            return 2
```

```
        if target == 0:
```

```
            return 1
```

```
        if nums[0] == target:
```

```
            return 1
```

```
    return 0
```

```
    if dp[i][target] != -1:
```

```
        return dp[i][target]
```

```
    notPick = self.solve(i - 1, target, nums, dp)
```

```
    pick = 0
```

```
    if nums[i] <= target:
```

```
        pick = self.solve(i - 1, target - nums[i], nums, dp)
```

```
    dp[i][target] = pick + notPick
```

```
    return dp[i][target]
```

```
def findTargetSumWays(self, nums, target):
```

```
    total = sum(nums)
```

```
    if abs(target) > total:
```

```
        return 0
```

```
    if (total - target) % 2 != 0:
```

```
        return 0
```

```
    target = (total - target) // 2
```

```
    n = len(nums)
```

```
    dp = [[-1] * (target + 1) for _ in range(n)]
```

```
    return self.solve(n - 1, target, nums, dp)
```

983. Minimum Cost For Tickets

```
class Solution:
```

```
    def mincostTickets(self, days: List[int], costs: List[int]) -> int:
```

```
        last_day = days[-1]
```

```
        dp = [0] * (last_day + 1)
```

```
        travel_days = set(days)
```

```
        for i in range(1, last_day + 1):
```

```
            if i not in travel_days:
```

```
                dp[i] = dp[i - 1]
```

```
            else:
```

```
cost1 = dp[max(0, i - 1)] + costs[0]
cost7 = dp[max(0, i - 7)] + costs[1]
cost30 = dp[max(0, i - 30)] + costs[2]
dp[i] = min(cost1, cost7, cost30)
```

```
return dp[last_day]
```

1049. Last Stone Weight II

class Solution:

```
def lastStoneWeightII(self, stones: List[int]) -> int:
```

```
    total = sum(stones)
```

```
    target = total // 2
```

```
    dp = [False] * (target + 1)
```

```
    dp[0] = True
```

```
    for stone in stones:
```

```
        for j in range(target, stone - 1, -1):
```

```
            dp[j] = dp[j] or dp[j - stone]
```

```
    for j in range(target, -1, -1):
```

```
        if dp[j]:
```

```
            return total - 2 * j
```